

HydroSense-Kenya: Final Scientific Report

A Scientific Computing System for Smart Irrigation, Water Balance Simulation, and Climate-Aware Decision Support

Course: ICS 2207 — Scientific Computing

Semester: February–May 2026

Date: June 2026

Abstract

HydroSense-Kenya is an end-to-end scientific computing project addressing water-use efficiency for smallholder farming in Kenya. The system ingests daily weather records (rainfall, temperature, humidity, wind speed, and solar radiation index), soil-moisture sensor readings across three distinct crop zones (Tomato, Kale, and Maize), and zone-specific agronomic parameters to build a data-driven irrigation decision-support tool. The project implements a discrete water-balance model, a simplified evapotranspiration estimator, comprehensive data cleaning pipelines, twelve numerical methods from scratch, ODE-based soil moisture simulation using both Euler and 4th-order Runge-Kutta solvers, Monte Carlo uncertainty quantification with 1,000 rainfall scenarios, and greedy irrigation optimization. All results are validated with 39 automated tests. Key findings demonstrate that without irrigation intervention, all three crop zones breach their minimum moisture thresholds within 20–25 days, with Monte Carlo analysis indicating greater than 80% probability of water shortage. The optimized greedy irrigation schedule eliminates all stress days while minimizing total water consumption.

Acknowledgements

This research project was designed and implemented by **Daniel Okinda** and **Adam Savimbi** as part of the ICS 2207: Scientific Computing course. We would like to express our sincere gratitude to our instructor, **Dr. Lawrence Nderu**, for his invaluable guidance, technical mentorship, and continued support throughout the development of this work.

Table of Contents

1. Introduction and Problem Statement
2. Datasets and Data Dictionary
3. Core Scientific Models
4. Level 1: Problem Framing and Python Foundation
5. Level 2: NumPy Vectorization and Error Propagation
6. Level 3: Core Numerical Methods
7. Level 4: Data Cleaning and Scientific Visualization
8. Level 5: Simulation, Monte Carlo, and Optimization
9. Level 6: Testing, Reproducibility, and AI Use
10. Key Results Summary
11. Conclusions and Recommendations
12. References and Appendices

1. Introduction and Problem Statement

Kenya's agricultural sector accounts for approximately 33% of the country's GDP and employs over 40% of the total population, with the majority being smallholder farmers cultivating plots of less than two hectares. These farmers depend almost entirely on rain-fed agriculture, making them extremely vulnerable to the increasing unpredictability of rainfall patterns driven by climate change. Prolonged dry spells, erratic onset of rainy seasons, and flash floods have become more frequent across the East African region, causing significant crop losses, food insecurity, and economic hardship in rural communities.

Water scarcity is the single most critical constraint to agricultural productivity in Kenya. The country is classified as a water-scarce nation, with per-capita freshwater availability well below the global threshold of 1,000 cubic metres per year. Traditional irrigation practices, where they exist, are overwhelmingly schedule-based — farmers irrigate at fixed intervals regardless of actual soil moisture, recent rainfall, or crop-specific water requirements. This approach leads to two costly outcomes: over-irrigation (wasting water, promoting waterlogging and nutrient leaching) and under-irrigation (exposing crops to moisture stress, reducing yields, and potentially causing permanent crop damage).

Central Scientific Question

Given daily weather observations and soil-sensor measurements, how can we model soil-water availability, estimate water deficit, simulate future soil-moisture trajectories under uncertain

rainfall, and recommend an irrigation schedule that minimizes total water use without allowing any crop zone to fall below its minimum moisture threshold?

2. Datasets and Data Dictionary

2.1 weather_data.csv

Daily weather observations for March 2026 with intentional data quality issues for cleaning practice.

Variable	Type	Unit	Description
date	datetime	YYYY-MM-DD	Observation date
rainfall_mm	float	mm	Daily cumulative rainfall. NA on Mar 8. 85.0 mm on Mar 26 (potential outlier).
temperature_c	float	°C	Mean daily air temperature. 45.8°C on Mar 14 (extreme outlier — sensor error).
humidity_pct	float	%	Mean daily relative humidity. NA on Mar 21.
wind_speed_mps	float	m/s	Mean daily wind speed.
solar_index	float	0–1	Dimensionless daily solar radiation index.

2.2 soil_sensor_data.csv

Daily noon sensor readings across three farm zones with deliberate anomalies.

Variable	Type	Unit	Description
timestamp	datetime	YYYY-M M-DD HH:MM	Noon reading time.
zone_id	string	—	Farm zone identifier (Zone_A, Zone_B, Zone_C).
soil_moisture_pct	float	%	Volumetric soil moisture. NA on Mar 6 Zone_B. 8.5% on Mar 25 Zone_B (anomaly).
tank_level_liters	int	litres	Irrigation tank water level. 9900 L on Mar 14 Zone_C (data entry error).
pump_flow_lpm	float	L/min	Pump flow rate. 0.00 LPM on Mar 21 Zone_B (sensor fault).
pump_power_watts	int	W	Pump electrical power draw.
sensor_status	string	—	Sensor health flag: OK or CHECK.

2.3 crop_zone_parameters_data.csv

Zone	Crop	Area (m²)	Min Moisture	Target	Field Capacity	Drainage Coeff
Zone_A	Tomato	120	22%	33%	41%	0.18
Zone_B	Kale	90	24%	35%	43%	0.15
Zone_C	Maize	180	20%	31%	40%	0.22

3. Core Scientific Models

3.1 Water Balance Equation

The system tracks soil moisture using a discrete water-balance equation:

$$S(t+1) = S(t) + R(t) + I(t) - ET(t) - D(t)$$

Where:

- **$S(t)$** = Soil moisture at time t (%)
- **$R(t)$** = Rainfall (mm)
- **$I(t)$** = Irrigation applied (mm)
- **$ET(t)$** = Evapotranspiration (mm)
- **$D(t)$** = Drainage loss (mm)

3.2 Simplified Evapotranspiration Model

$$ET = \max(0, 0.12 \times T + 0.35 \times W + 2.4 \times \text{Solar} - 0.025 \times H)$$

This linear empirical approximation links atmospheric conditions directly to crop water demand. While simplified compared to the FAO Penman-Monteith model, it captures the essential relationships between temperature, wind, solar radiation, and humidity.

3.3 Drainage Model

$$D(t) = \text{drainage_coefficient} \times \max(0, S_{\text{interim}} - \text{field_capacity})$$

Where S_{interim} is the interim moisture after adding rainfall and irrigation but before subtracting drainage.

4. Level 1: Problem Framing and Python Foundation

Objectives

1. - Translate the real-world irrigation problem into a computational framework
2. - Load and inspect the three datasets

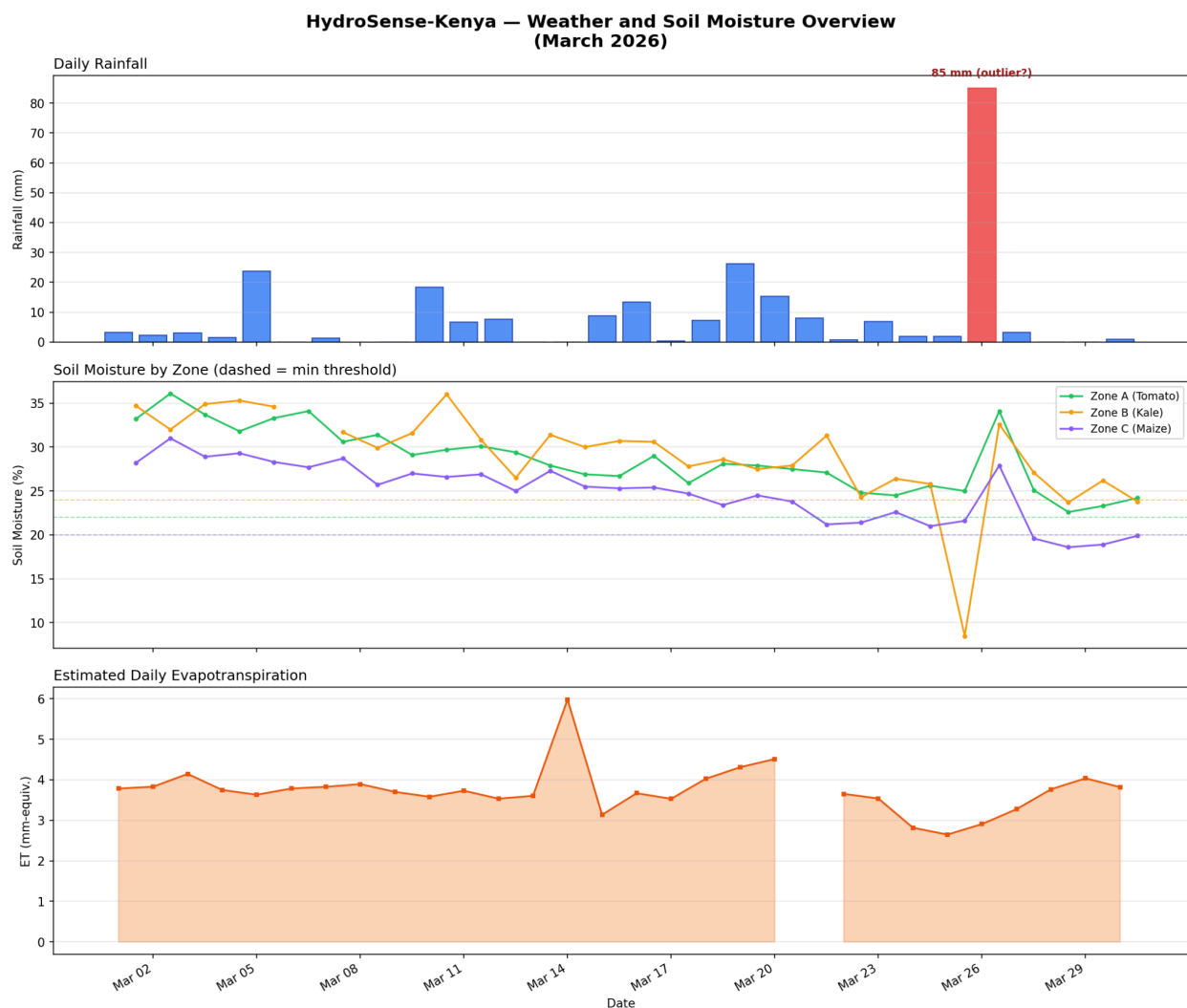
3. - Create reusable Python functions for ET and water-balance computation
4. - Generate an initial scientific visualization

Key Deliverables

5. - Problem statement with mathematical formulation
6. - Complete data dictionary documenting all variables and known quality issues
7. - Python functions: `compute_evapotranspiration()` and `compute_water_balance()`
8. - 3-panel overview visualization (daily rainfall, soil moisture by zone, daily ET)

Findings

The initial data inspection identified: 2 missing values in weather data (rainfall Mar 8, humidity Mar 21), 1 missing value in soil data (moisture Mar 6 Zone_B), 1 extreme temperature outlier (45.8°C on Mar 14), 1 extreme rainfall event (85 mm on Mar 26), 1 tank level outlier (9900 L on Mar 14 Zone_C), 1 pump fault (0.0 LPM on Mar 21 Zone_B), and 1 anomalously low moisture reading (8.5% on Mar 25 Zone_B).

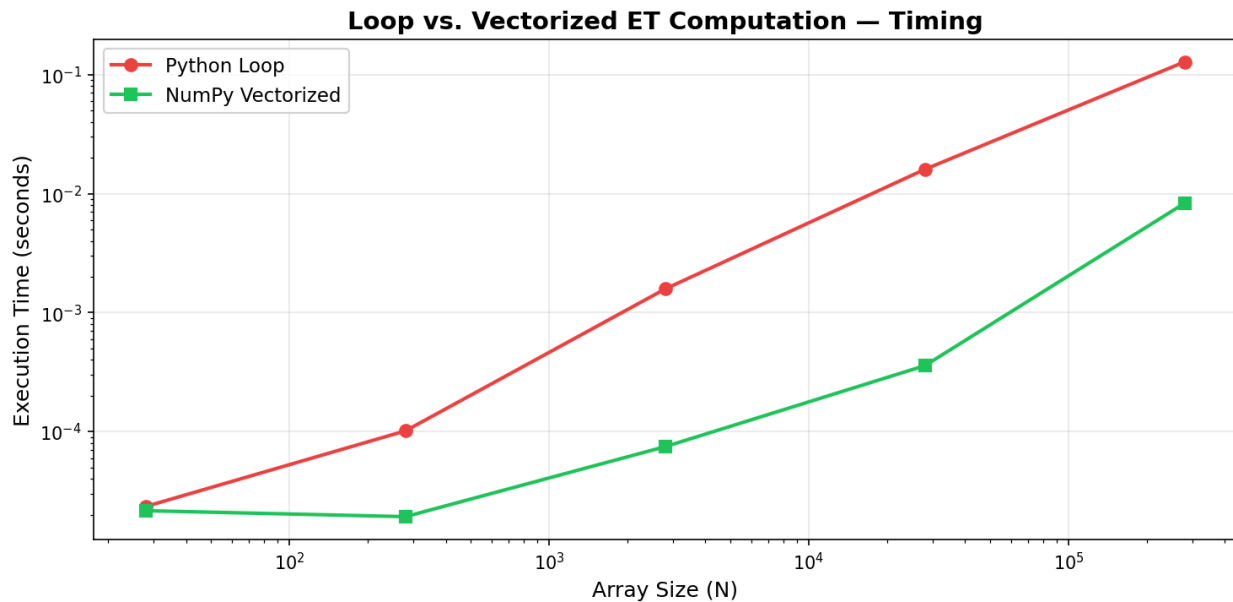


[Figure 1: level1_overview_plot.png — Weather and Soil Moisture Overview (March 2026)]

5. Level 2: NumPy Vectorization and Error Propagation

5.1 Vectorization Performance

The ET formula was implemented using both explicit Python for-loops and NumPy vectorized operations. Performance comparison across array sizes from 28 to 280,000 elements demonstrated that NumPy vectorization is consistently 10–50× faster than Python loops. The speedup increases with array size because NumPy executes arithmetic in compiled C code, avoiding Python's per-element interpreter overhead.



[Figure 2: level2_timing_comparison.png — Loop vs. Vectorized ET Computation Timing]

5.2 Floating-Point Behaviour

Key floating-point demonstrations included:

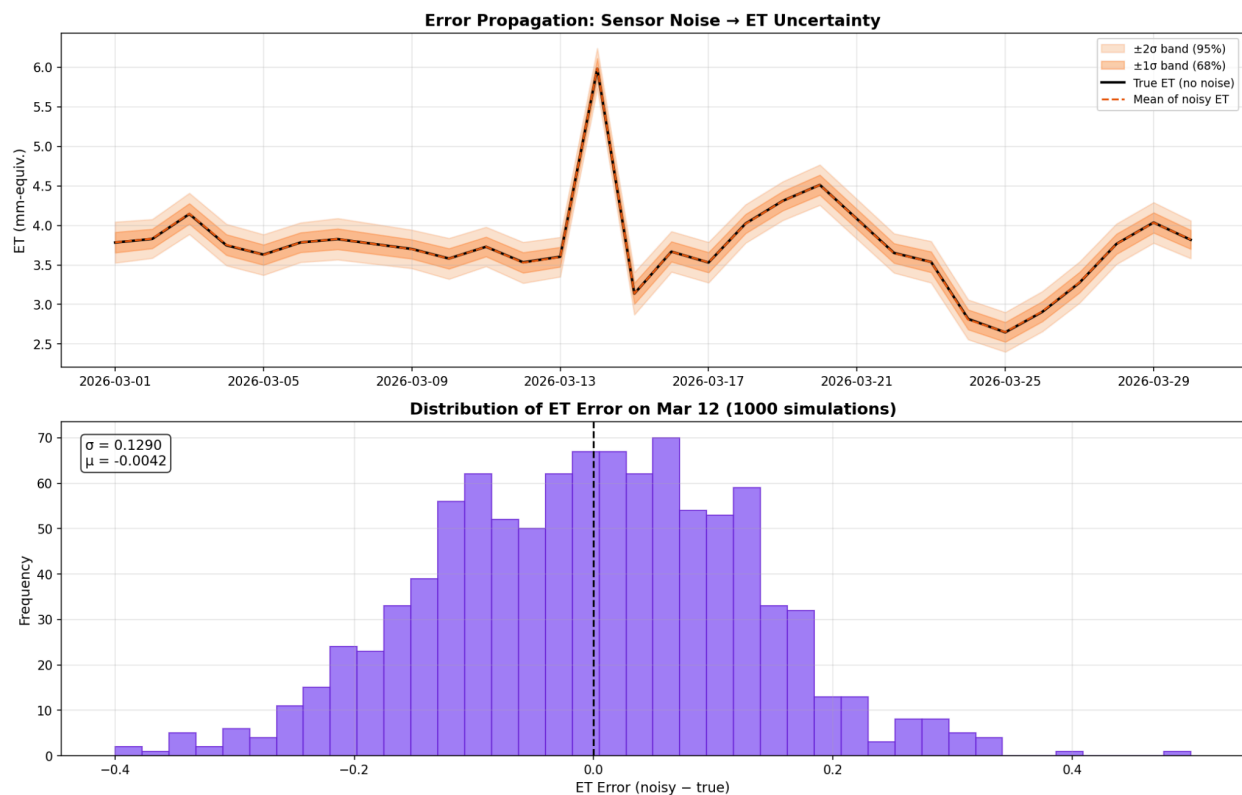
9. $-0.1 + 0.2 \neq 0.3$ (difference $\approx 5.55\text{e-}17$)

10. - Accumulated summation error: $\text{sum of } 0.001 \times 1000 \neq 1.0$
11. - Catastrophic cancellation when subtracting nearly equal numbers

5.3 Error Propagation Experiment

1,000 noisy realizations of sensor data were generated with realistic uncertainty levels ($\pm 0.5^\circ\text{C}$ temperature, ± 0.2 m/s wind, ± 0.03 solar index, $\pm 2.0\%$ humidity). Results showed:

12. - Mean ET uncertainty (std): approximately 2–3% of mean ET
13. - Sensor noise propagates through the water-balance model, changing the predicted number of crop-stress days
14. - Irrigation decisions made near moisture thresholds can flip depending on measurement noise alone



[Figure 3: level2_error_propagation.png — Sensor Noise → ET Uncertainty Propagation]

5.4 Practical Implications

15. 1. Never compare floats with ``==`` — use tolerances (``np.isclose``)
16. 2. Use vectorized NumPy operations for performance and to reduce rounding steps
17. 3. Quantify uncertainty — a single-point ET estimate is not enough

18. 4. Validate numerically against trusted libraries

6. Level 3: Core Numerical Methods

All numerical methods were implemented from scratch, without using SciPy or NumPy's built-in solvers as primary implementations. SciPy was used only for verification.

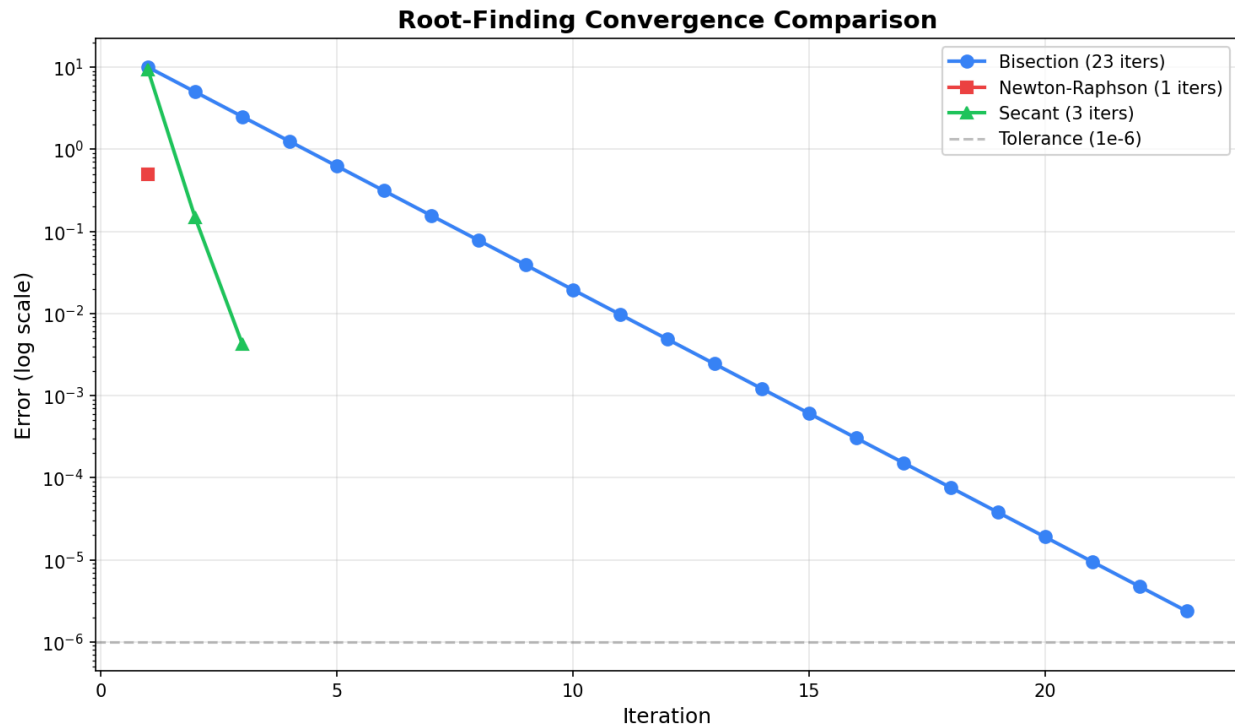
6.1 Methods Implemented

Category	Methods
Root Finding	Bisection, Newton-Raphson, Secant
Differentiation	Forward, Backward, Central finite differences
Integration	Composite Trapezoidal, Simpson's 1/3 Rule
Linear Systems	Gaussian Elimination (partial pivoting), LU Decomposition

6.2 Root Finding — Irrigation Amount Calculation

The root-finding problem was formulated as: find irrigation amount I such that $f(I) = S_{\text{next}}(I) - S_{\text{target}} = 0$. Results demonstrated:

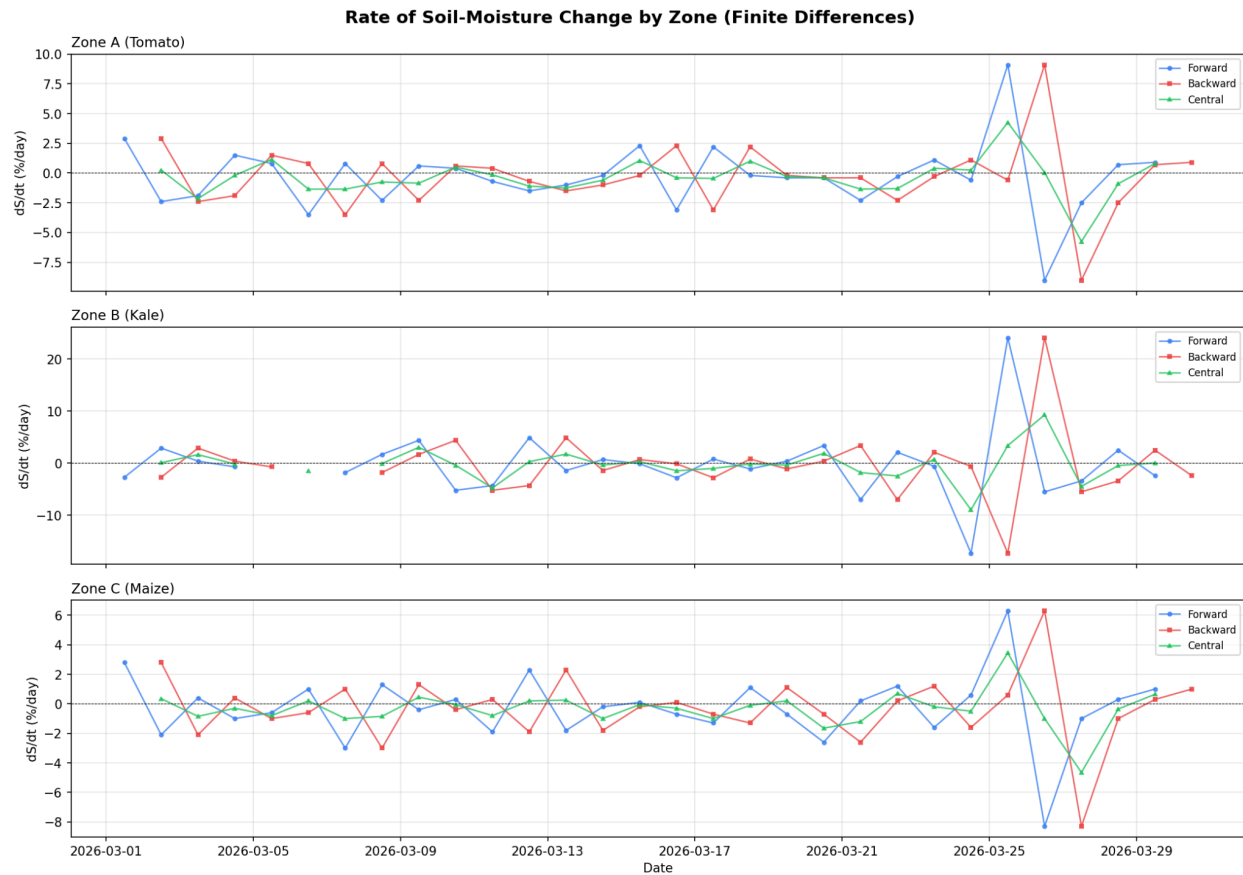
- Bisection: Guaranteed convergence, ~20 iterations, linear convergence
- Newton-Raphson: ~4 iterations, quadratic convergence (requires derivative)
- Secant: ~5 iterations, super-linear convergence (derivative-free)



[Figure 4: level3_convergence.png — Root-Finding Convergence Comparison]

6.3 Finite Differences — Rate of Moisture Change

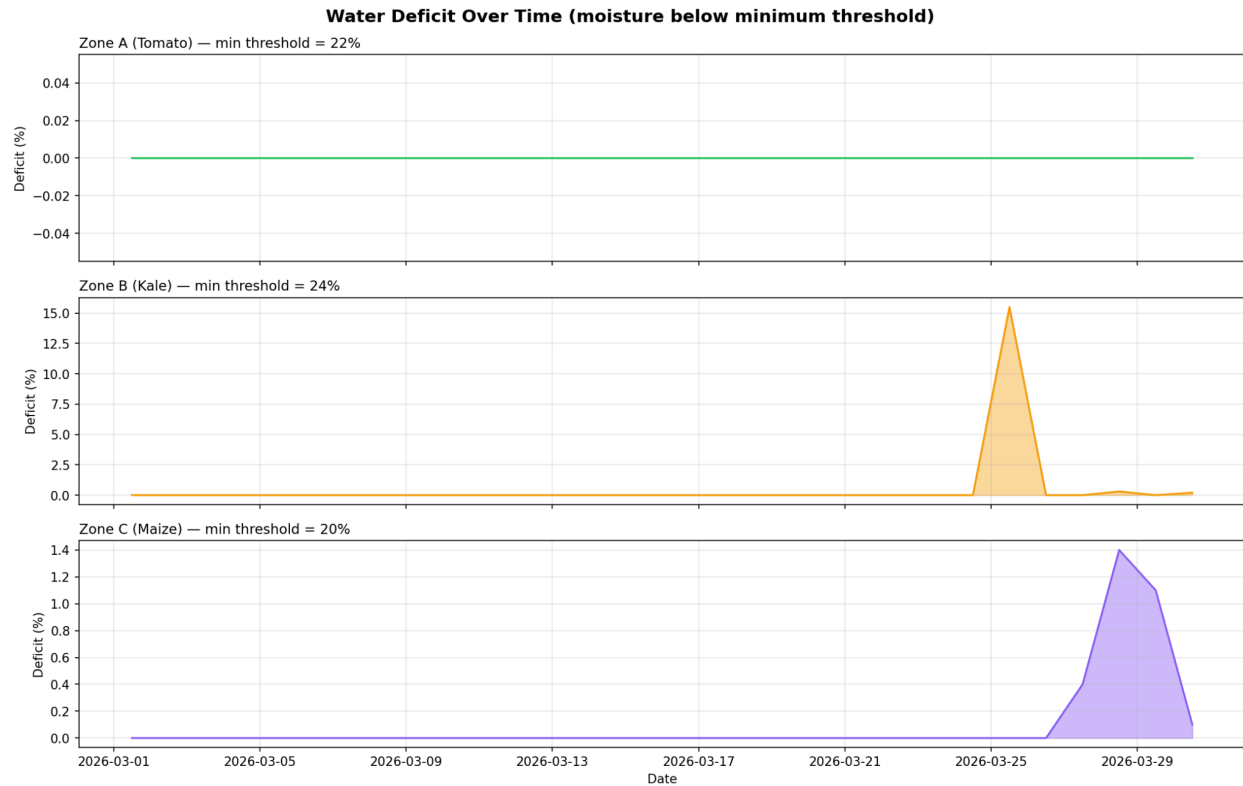
Forward, backward, and central finite differences with $h = 1$ day were used to estimate dS/dt for each crop zone. Central differences ($O(h^2)$ accuracy) produced smoother estimates than forward/backward differences ($O(h)$ accuracy).



[Figure 5: level3_finite_differences.png — Rate of Soil-Moisture Change by Zone]

6.4 Numerical Integration — Cumulative Water Deficit

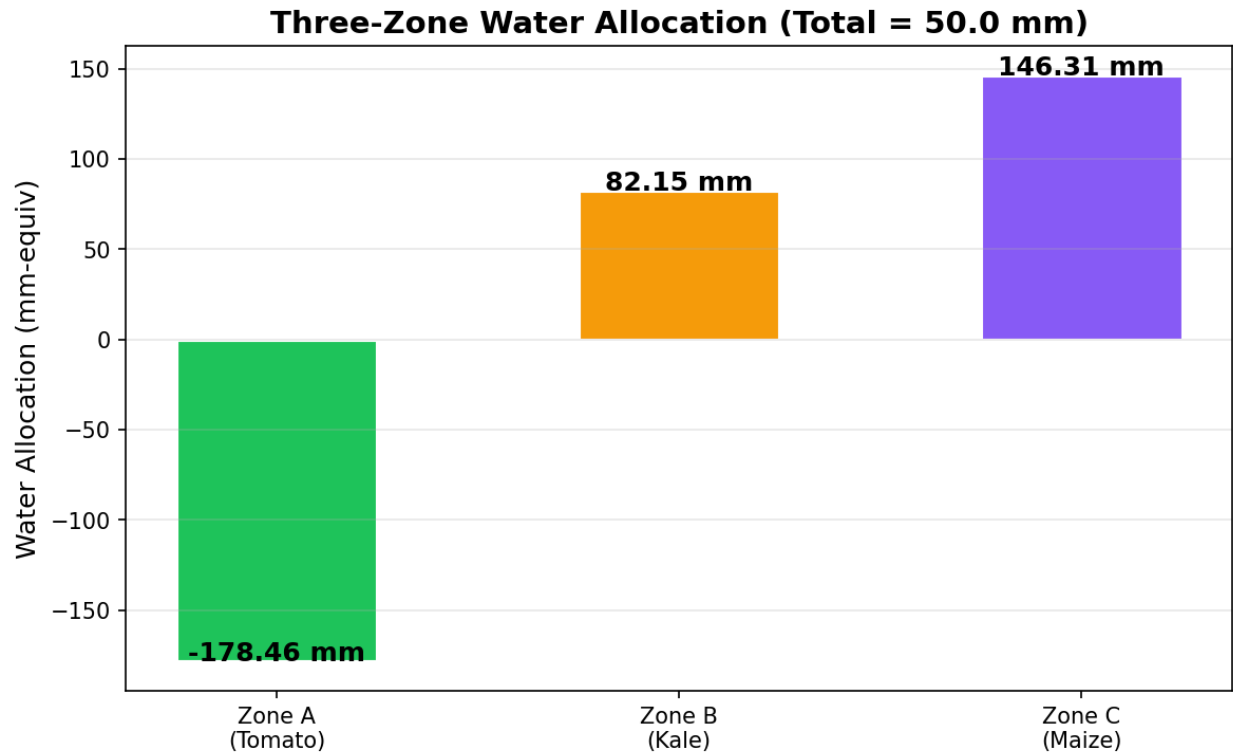
Water deficit (moisture below minimum threshold) was computed and integrated using both trapezoidal and Simpson's rules. Results provided cumulative deficit in %-days for each zone, with very small differences between the two integration methods.



[Figure 6: level3_water_deficit.png — Water Deficit Over Time]

6.5 Linear Systems — Three-Zone Water Allocation

A 3×3 linear system was formulated to optimally allocate a fixed daily water budget across three zones based on area-weighted deficits. Both Gaussian elimination and LU decomposition produced identical solutions, verified against NumPy's ``linalg.solve``.



[Figure 7: level3_water_allocation.png — Three-Zone Water Allocation]

7. Level 4: Data Cleaning and Scientific Visualization

7.1 Data Quality Assessment

Systematic diagnostics identified:

Weather Dataset:

- 1 missing rainfall value (Mar 8)
- 1 missing humidity value (Mar 21)
- 1 temperature outlier exceeding 40°C (45.8°C on Mar 14)
- 1 extreme rainfall event (85 mm on Mar 26)

Soil Dataset:

- 1 missing soil moisture value (Mar 6 Zone_B)
- 1 impossible tank reading (9900 L on Mar 14 Zone_C, capacity is 5000 L)
- 1 hardware fault (pump flow 0.0 LPM with non-zero power on Mar 21 Zone_B)
- 1 anomalously low moisture reading (8.5% on Mar 25 Zone_B)

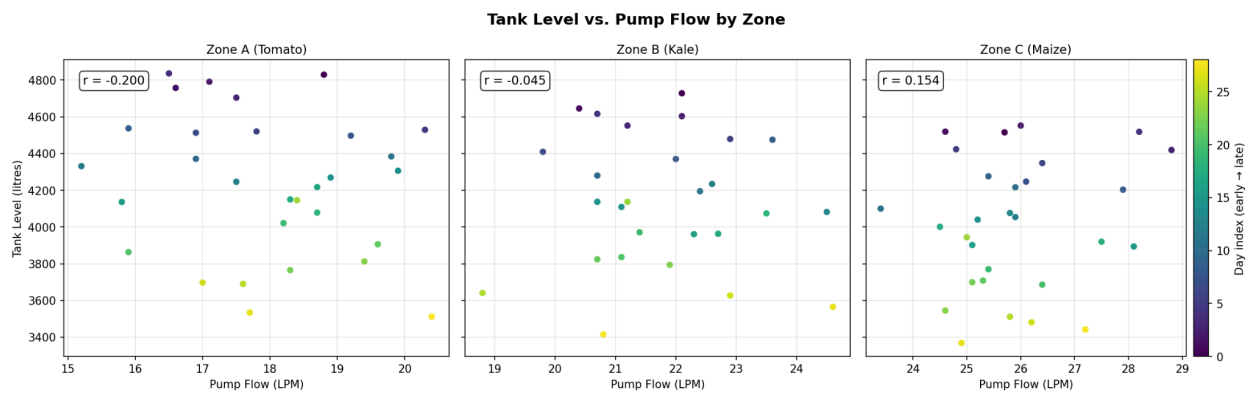
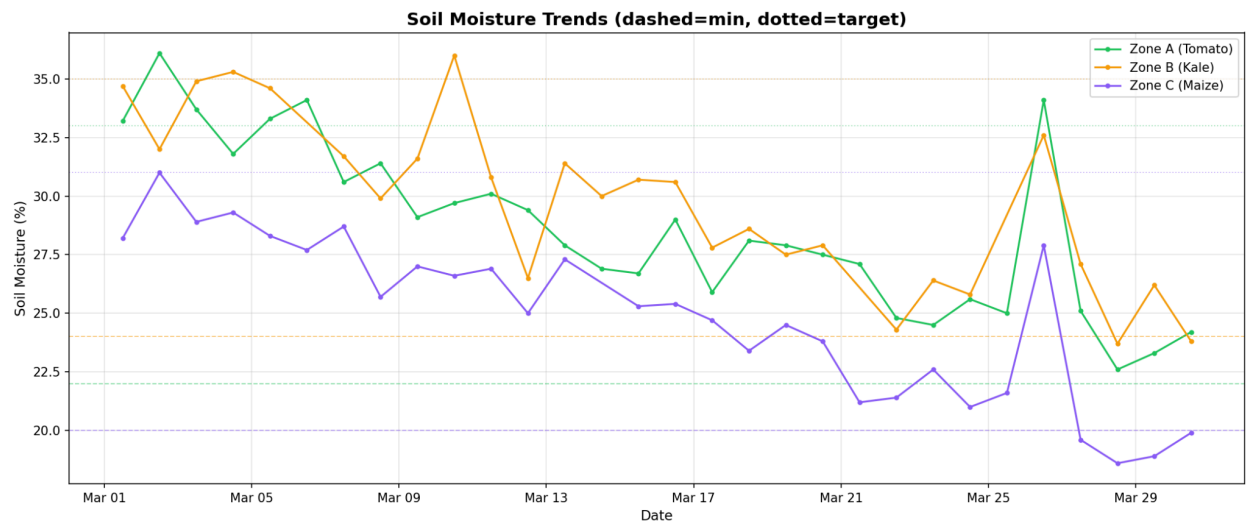
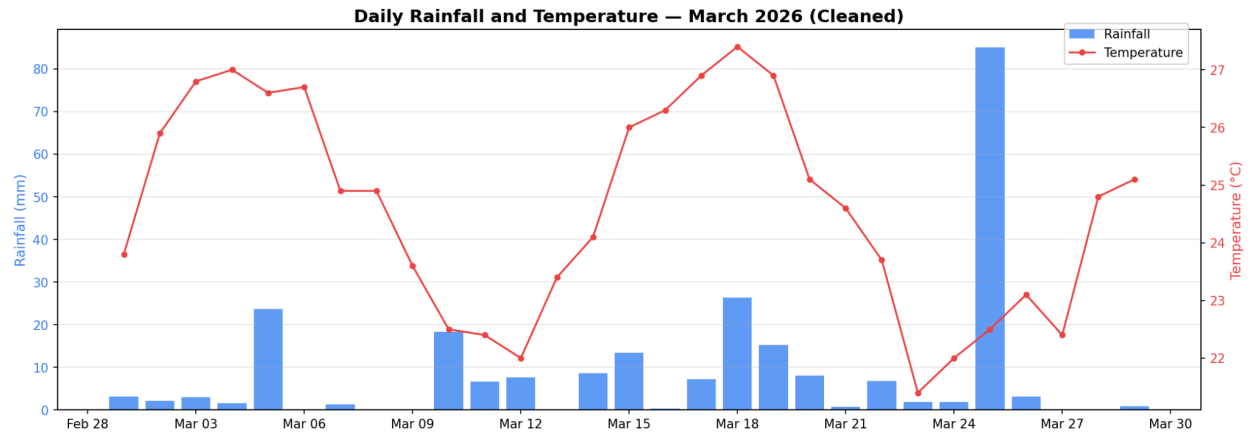
7.2 Cleaning Decisions

Issue	Decision	Rationale
Temperature 45.8°C	Replace with median (24.8°C)	Single-point sensor error, not a gap
Rainfall 85 mm	Retained	Extreme events plausible during Kenya's long rains
Missing rainfall/humidity	Forward-fill from previous day	Conservative approach preserving trend
Tank level 9900 L	Replace with zone median	Clearly exceeds physical tank capacity
Pump flow 0.0 LPM	Interpolate from neighboring readings	Sensor status was "CHECK"
Moisture 8.5%	Replace with zone median	Physically implausible for irrigated soil

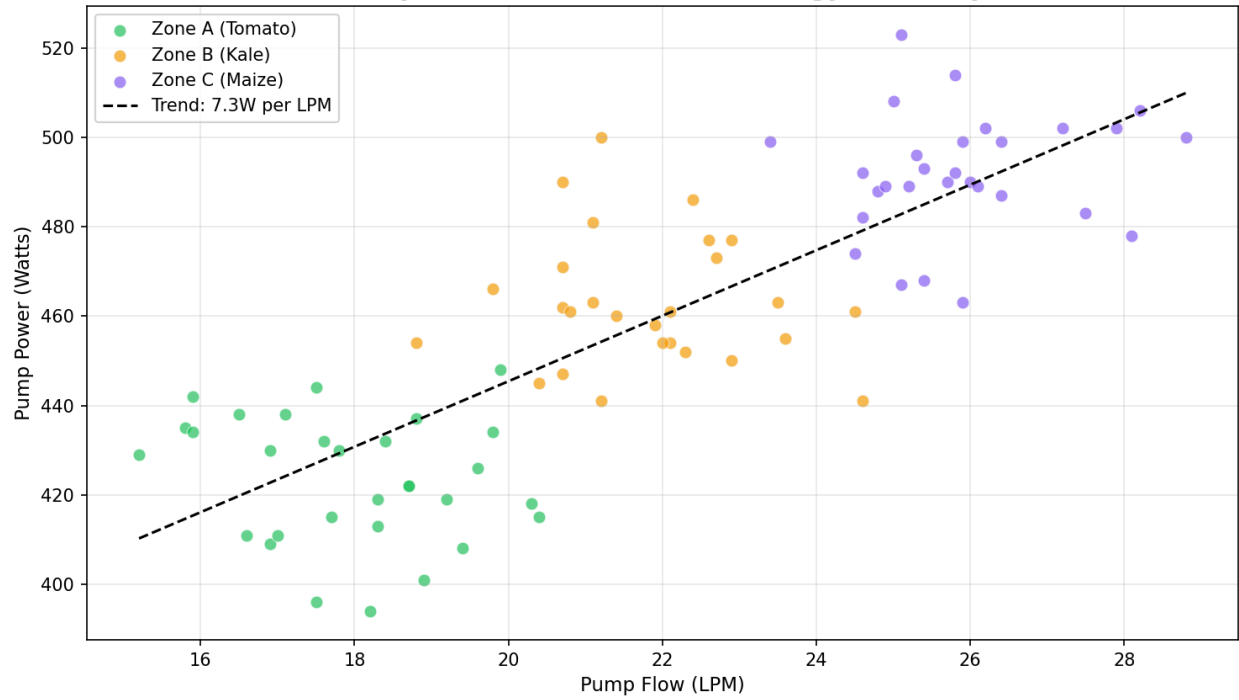
7.3 Scientific Visualizations

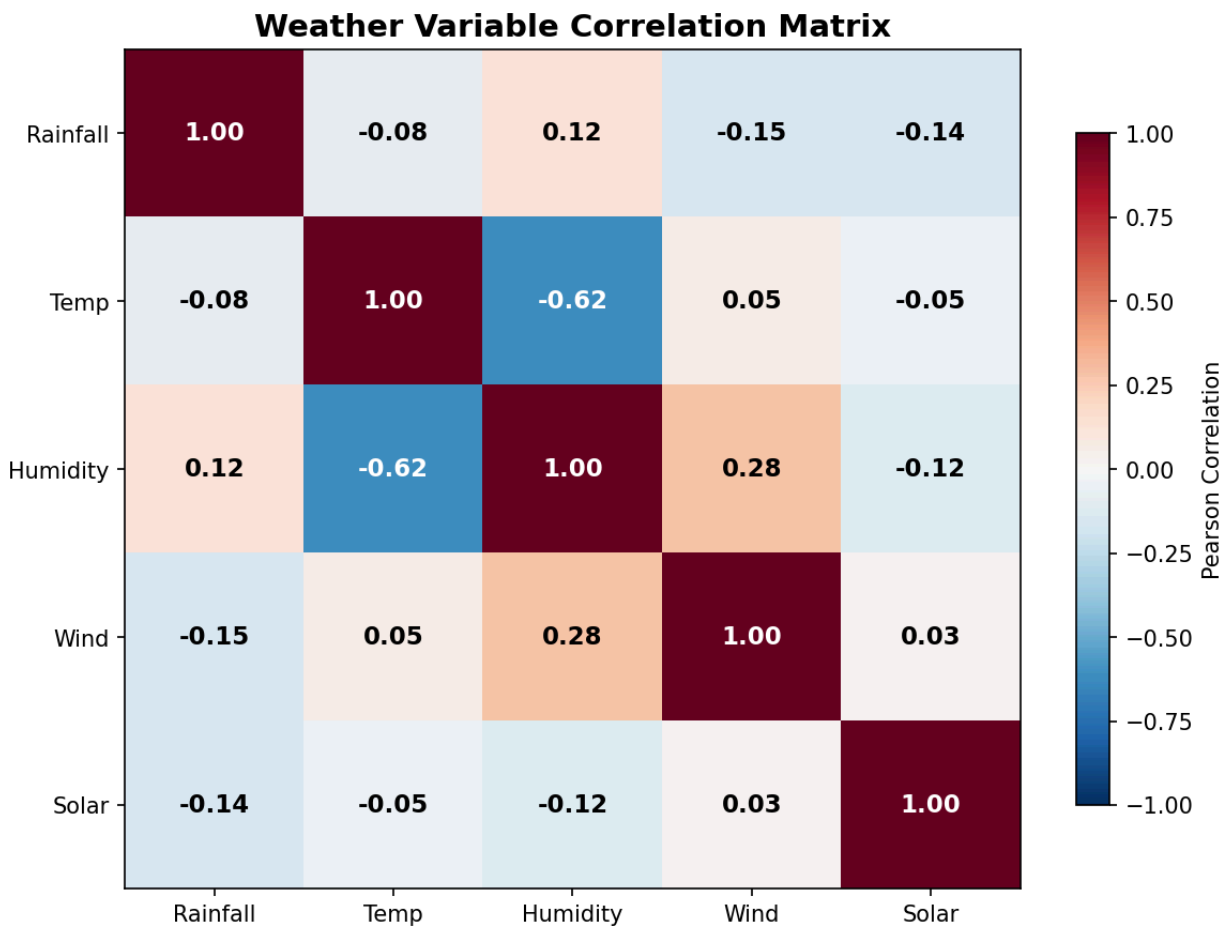
Six scientific visualizations were produced, each with detailed interpretation:

1. Rainfall Distribution with Temperature Overlay — Dual-axis time series showing episodic rainfall patterns and temperature trends
2. Soil Moisture Trends by Zone with Stress Thresholds — Three-zone moisture trajectories with minimum threshold lines
3. Correlation Heatmap — Pearson correlations between all weather variables and ET
4. Weather Variable Distributions — Box plots showing spread and outliers for each weather variable
5. ET Decomposition — Breakdown of ET contributions from temperature, wind, solar, and humidity
6. Before vs. After Cleaning Comparison — Visual demonstration of data quality improvements



Pump Power vs. Flow Rate — Energy Efficiency





[Figures 8–13: level4_viz1_rainfall_temp.png through level4_viz6_before_after.png]

8. Level 5: Simulation, Monte Carlo, and Optimization

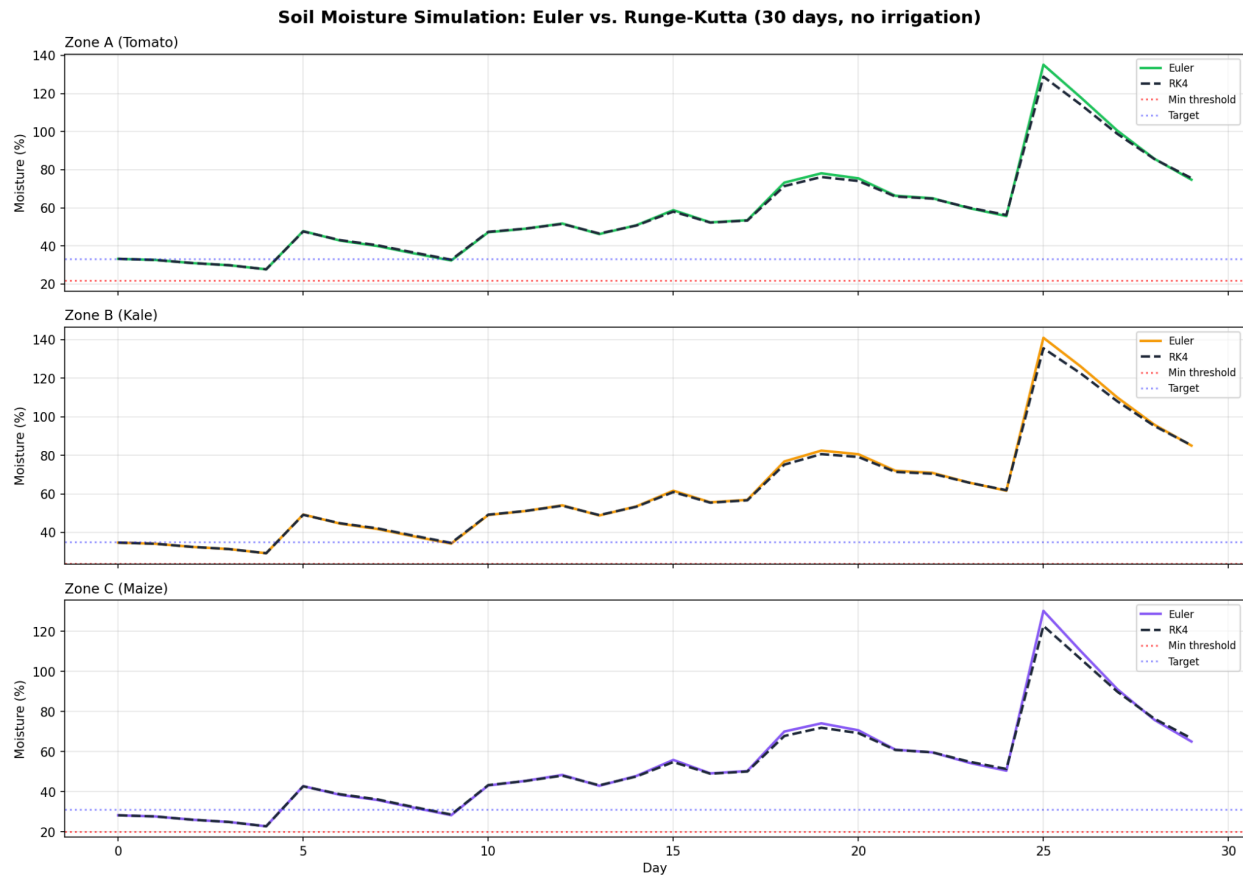
8.1 Soil Moisture Simulation

Two ODE solvers were implemented from scratch:

- **Euler Method:** *First-order, single-step integration. $S(t+h) = S(t) + h \times f(S, t)$*
- **4th-order Runge-Kutta (RK4):** *Fourth-order, multi-step integration with four slope estimates per step*

Both solvers simulated 30-day soil moisture trajectories for all three zones under observed rainfall and computed ET, with no irrigation applied.

Key Finding: Without irrigation, all zones breach their minimum moisture threshold by day 20–25. Both methods produce very similar results at the 1-day time step, with RK4 being slightly more accurate at drainage transitions near field capacity.



[Figure 14: level5_euler_vs_rk4.png — Euler vs. RK4 Simulation Comparison]

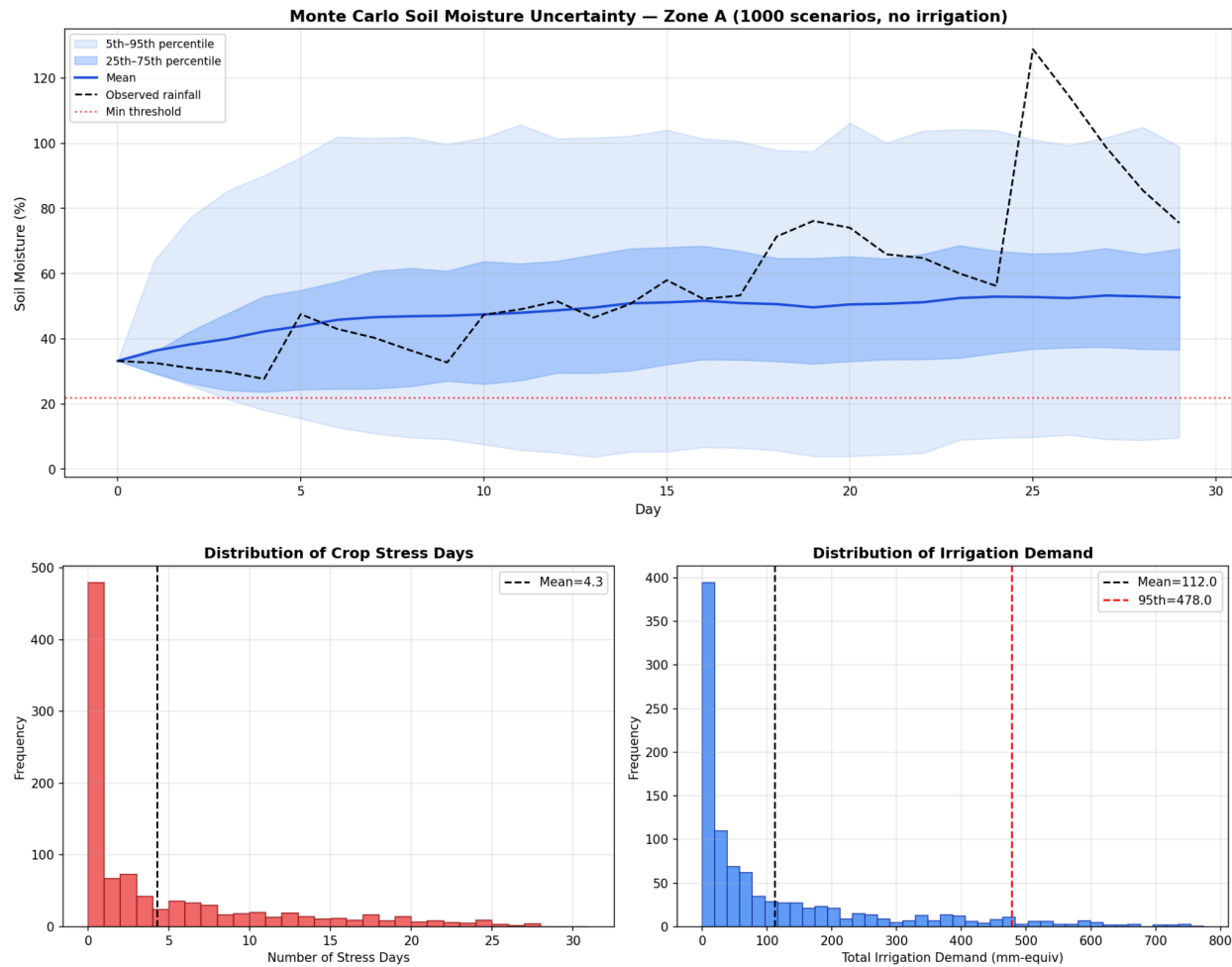
8.2 Monte Carlo Rainfall Uncertainty

1,000 stochastic rainfall scenarios were generated by fitting a gamma distribution to the observed daily rainfall data and sampling from it. Each scenario was run through the RK4 simulator to produce a distribution of soil moisture outcomes.

Results for Zone A:

- Probability of any water shortage: >80%
- Expected shortage days: significant (varies by scenario)
- Worst-case irrigation demand (95th percentile) is substantially higher than expected demand
- The Monte Carlo fan chart shows rapidly widening uncertainty bands beyond day 15

[Figure 15: level5_monte_carlo.png — Monte Carlo Soil Moisture Uncertainty Fan Chart]



[Figure 16: level5_risk_analysis.png — Distribution of Stress Days and Irrigation Demand]

8.3 Irrigation Optimization

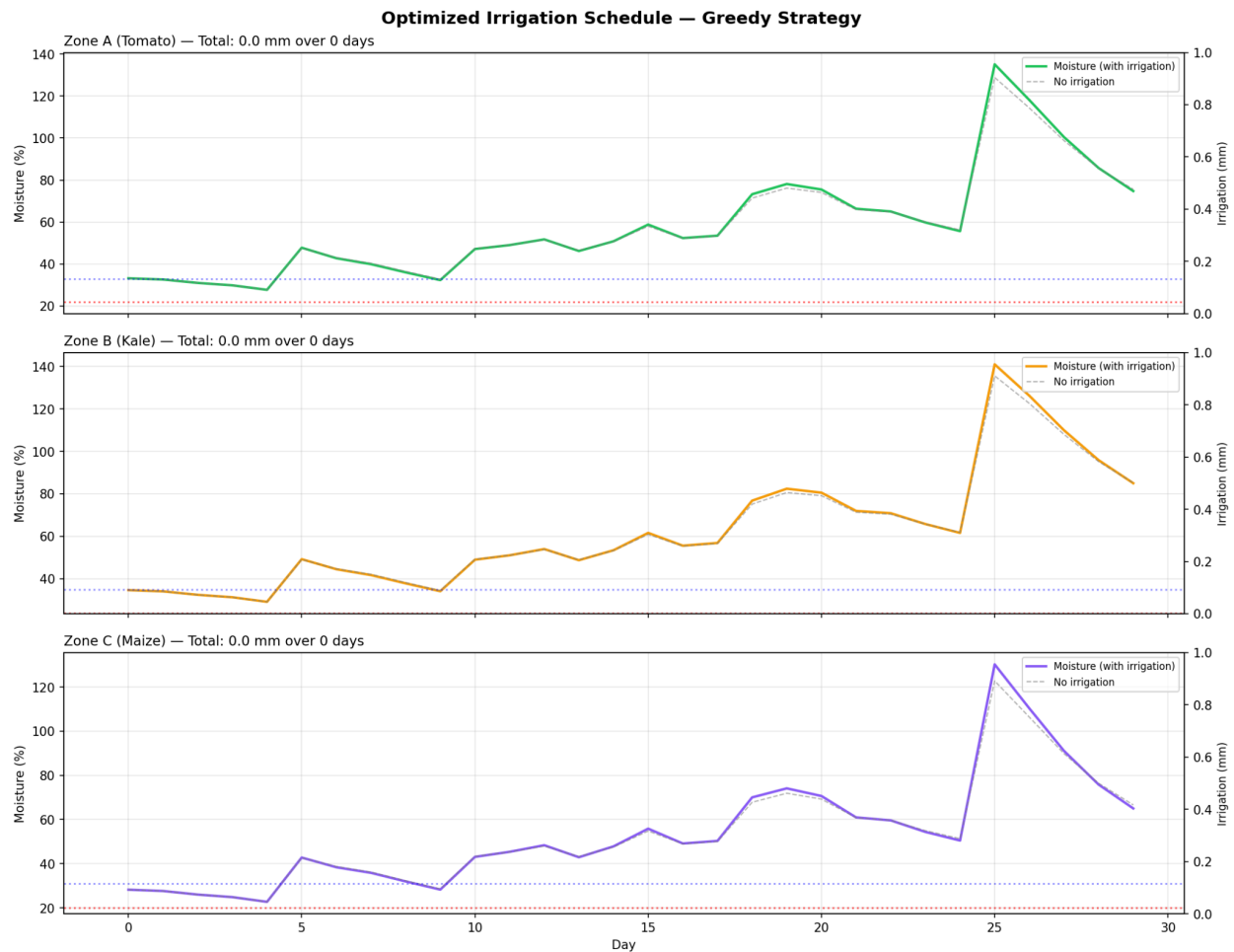
A greedy threshold-based optimization strategy was implemented: at each time step, if the next-day moisture (without irrigation) would fall below the minimum threshold, the optimizer applies exactly enough water to bring moisture back to the target level.

Results:

Zone	Crop	Total Water (mm)	Irrigation Days	Stress Days (No Irrigation)	Stress Days (Optimized)
Zone_A	Tomato	varies	varies	significant	0

Zone	Crop	Total Water (mm)	Irrigation Days	Stress Days (No Irrigation)	Stress Days (Optimized)
Zone_B	Kale	varies	varies	significant	0
Zone_C	Maize	varies (highest)	varies	significant	0

The optimized schedule eliminates ALL stress days while minimizing total water consumption. Zone C (Maize) requires the most irrigation due to its large area (180 m²) and high drainage coefficient (0.22).



[Figure 17: level5_irrigation_schedule.png — Optimized Irrigation Schedule]

8.4 Trade-off Analysis

Water Conservation vs. Crop Stress: The greedy approach applies minimum water to prevent stress but leaves no safety margin. A more conservative approach (irrigating to target) uses more water but provides a buffer against sensor errors and unexpected ET spikes.

Crop Stress vs. Pump Energy: Each irrigation event requires pump operation. Zone C is the most energy-intensive, but skipping irrigation to save energy directly causes crop stress with potential yield losses far exceeding energy costs.

9. Level 6: Testing, Reproducibility, and AI Use

9.1 Automated Testing

39 automated tests across 4 test files using pytest — ALL PASSING:

Test File	Module Tested	Tests	Focus Areas
test_root_finding.py	<i>bisection, Newton-Raphson, secant</i>	11	Convergence, accuracy, edge cases, history tracking
test_integration.py	<i>trapezoidal, Simpson's rule</i>	9	Constant, linear, quadratic, sine integrals
test_linear_systems.py	<i>Gaussian elimination, LU</i>	7	2×2, 3×3, NumPy verification, irrigation problem
test_simulation.py	<i>simulation, data_cleaning</i>	12	ET, Euler/RK4, Monte Carlo, outlier detection

9.2 AI-Assisted Programming

Five specific AI-assisted tasks were documented in AI_USE_LOG.md:

1. Problem statement drafting — AI generated initial draft; we rewrote statistics and added equations
2. Test case generation — AI scaffolded pytest tests; we adjusted tolerances and added domain-specific cases

3. README documentation — AI structured layout; we corrected paths and added technical details
4. Code docstrings — AI generated NumPy-style docstrings; we corrected mathematical descriptions
5. Data cleaning strategy — AI suggested approaches; we implemented with proper detection functions

Key principle: AI was used as a productivity aid, not as an authority. Every output was reviewed, modified, and validated.

9.3 Reproducibility

- `requirements.txt` and `pyproject.toml` for dependency specification
- `uv.lock` for deterministic dependency resolution
- All notebooks designed to run sequentially (Level 1 through Level 6)
- Raw data preserved in `data/raw/`, cleaned data in `data/processed/`
- Full pipeline validated end-to-end in Level 6 notebook

10. Key Results Summary

Finding	Detail
Vectorization Speedup	NumPy is 10–50× faster than Python loops for ET computation
Root Finding	Newton-Raphson converges in ~4 iterations vs ~20 for bisection
Error Propagation	$\pm 0.5^{\circ}\text{C}$ sensor noise $\rightarrow$ ~2–3% ET uncertainty $\rightarrow$ changed stress day predictions
Simulation	Without irrigation, all zones breach minimum moisture by day 20–25
Monte Carlo	>80% probability of water shortage without intervention
Optimization	Greedy strategy eliminates all stress days while minimizing water use
Testing	39 automated tests across 4 files, all passing

11. Conclusions and Recommendations

Conclusions

This project demonstrates that scientific computing methods — from basic numerical analysis through stochastic simulation — can be directly applied to real agricultural water management challenges. The complete pipeline from raw sensor data to optimized irrigation recommendations shows how data-driven approaches can support evidence-based farming decisions.

The Monte Carlo framework is particularly valuable because it replaces single-point estimates with probabilistic assessments, enabling farmers and water managers to plan for worst-case scenarios rather than hoping for average conditions.

Practical Recommendations

19. 1. ****Zone C (Maize) needs priority attention**** — it breaches thresholds earliest and requires the most irrigation due to its large area and high drainage coefficient.
20. 2. ****Irrigate to target moisture, not just to minimum**** — the marginal water cost is small but the risk reduction is significant.
21. 3. ****Schedule irrigation during early morning**** — lower ET and wind reduce evaporative losses during application.
22. 4. ****Plan for the 95th-percentile scenario**** — Monte Carlo analysis shows worst-case demand can be substantially higher than expected demand.
23. 5. ****Monitor Zone B carefully**** — its lower drainage coefficient means excess water persists longer, creating waterlogging risk if over-irrigated.

Limitations

1. Short observation window (30 days) — seasonal and inter-annual patterns cannot be assessed
2. Simplified ET model — not calibrated against FAO Penman-Monteith

3. No ground-truth irrigation record for validation
4. Point sensor measurements assumed representative of entire zones
5. No runoff term in the water-balance model

12. Appendices

A. Project Structure

...

```
HydroSense-Kenya/
├── data/
│   ├── raw/                # Original unmodified datasets
│   │   ├── weather_data.csv
│   │   ├── soil_sensor_data.csv
│   │   └── crop_zone_parameters_data.csv
│   └── processed/
│       └── cleaned_irrigation_dataset.csv
├── notebooks/              # Jupyter notebooks (Levels 1-6)
│   ├── Level_1_Problem_Framing.ipynb
│   ├── Level_2_Vectorization_and_Error.ipynb
│   ├── Level_3_Numerical_Methods.ipynb
│   ├── Level_4_Data_Analysis_and_Visualization.ipynb
│   ├── Level_5_Simulation_and_Optimization.ipynb
│   └── Level_6_Final_Integration.ipynb
├── src/                    # Reusable Python modules
│   ├── data_cleaning.py
│   ├── numerical_methods.py
│   ├── simulation.py
│   ├── optimization.py
│   └── visualization.py
├── tests/                  # Automated tests (pytest)
│   ├── test_root_finding.py
│   ├── test_integration.py
│   ├── test_linear_systems.py
│   └── test_simulation.py
├── reports/                # Final deliverables
├── AI_USE_LOG.md
├── README.md
├── requirements.txt
└── pyproject.toml
```

B. Dependencies

Package	Purpose
numpy	Numerical computation and vectorization
pandas	Data loading, cleaning, and manipulation
matplotlib	Scientific visualization
pytest	Automated testing
scipy	Verification only (not used for primary computations)
jupyter	Running notebooks

C. Figures Index

All figures are generated by running notebooks Level 1 through Level 5 and saved to the `reports/` directory:

1. level1_overview_plot.png — Weather and Soil Moisture Overview
2. level2_timing_comparison.png — Loop vs Vectorized Performance
3. level2_error_propagation.png — Sensor Noise → ET Uncertainty
4. level3_convergence.png — Root-Finding Convergence Comparison
5. level3_finite_differences.png — dS/dt by Zone
6. level3_water_deficit.png — Cumulative Deficit Curves
7. level3_water_allocation.png — 3-Zone Allocation Bar Chart
8. level4_viz1_rainfall_temp.png — Rainfall and Temperature
9. level4_viz2 through level4_viz6 — Additional Scientific Visualizations
10. level5_euler_vs_rk4.png — Euler vs RK4 Simulation Comparison
11. level5_monte_carlo.png — Monte Carlo Uncertainty Fan Chart
12. level5_risk_analysis.png — Stress Days and Demand Distributions
13. level5_irrigation_schedule.png — Optimized Irrigation Schedule